



Faculty of Engineering and Technology
Department of Electrical and Computer Engineering
ENCS5141 — Intelligent Systems Laboratory

Assignment #1

**Data Preprocessing, Feature Engineering and Comparative
Classification on the Heart Disease Dataset**

Prepared by: Tawba Abdallah—1221002

Instructor: Aziz Qaroush

Assistant: Rami Majadbeh

Date: May 15, 2026

Abstract

This report describes a full machine learning project that was done on the Heart Disease dataset. The dataset has 918 patient records and 12 different pieces of information about each patient, such as their age, blood pressure, and cholesterol level. The goal of the project was to build machine learning models that can predict whether or not a patient has heart disease based on these measurements.

The work is split into two main parts. In Part 1, we look closely at the raw data and try to find any problems with it. I found three main problems that are specific to this dataset. First, the Sex column had 10 rows with invalid codes like "X", "Z", and "G" that are clearly not real gender values. Second, the Cholesterol and RestingBP columns had values of 0 in many rows, which is medically impossible for a living person — this means those were actually missing values that were written as 0 by mistake. Third, there were 13 rows where the Oldpeak column had negative values, which also does not make medical sense. I built a cleaning pipeline to fix all of these problems, and then compared a Random Forest model trained on the dirty raw data versus the clean preprocessed data.

In Part 2, I trained and compared three different machine learning models on the cleaned data: a Random Forest (RF), a Support Vector Machine (SVM), and a Multi-Layer Perceptron (MLP). I first ran all three models with their default settings, then used a technique called GridSearchCV to find the best settings for each one. After tuning, the SVM model got the best result with an F1 score of 0.854 and a ROC-AUC of 0.918. The Random Forest got an F1 of 0.844, and the MLP got 0.834. One interesting and surprising finding was that the Random Forest model trained on the raw data actually performed better than the one trained on the preprocessed data. I explain why this happened in the discussion section.

Contents

Abstract.....	I
Contents.....	II
1. Introduction.....	1
2. Dataset Description.....	2
2.1 Hidden Data Quality Problems.....	3
3. Procedure.....	6
3.1 Part 1: Data Preprocessing and Feature Engineering.....	6
3.1.1 Step 1 — Data Cleaning.....	6
3.1.2 Step 2 — Encoding Categorical Variables.....	8
3.1.3 Step 3 — Train/Test Split and Feature Scaling.....	9
3.1.4 Step 4 — Feature Selection.....	10
3.1.5 Step 5 — Handling Class Imbalance with SMOTE.....	13
3.2 Part 2: Comparative Analysis of Classification Models.....	14
3.2.1 The Three Models.....	14
3.2.2 Evaluation Metrics.....	15
3.2.3 Hyper-Parameter Tuning with GridSearchCV.....	15
4. Results.....	18
4.1 Part 1: Random Forest — Raw Data vs. Preprocessed Data.....	18
4.2 Part 2: Default Models on Preprocessed Data.....	18
4.3 Part 2: Tuned Models After GridSearchCV.....	19
4.4 Computational Cost Comparison.....	21
5. Discussion.....	23
5.1 Why Did the Raw Data Model Beat the Preprocessed Model?.....	23
5.2 Comparing the Three Models.....	24
5.3 Practical Recommendation.....	24
6. Conclusion.....	26
References.....	27

1. Introduction

Heart disease is one of the most serious health problems in the world. According to the World Health Organization, it causes about 17.9 million deaths every year, which makes it the number one cause of death globally. Because of this, researchers and doctors are always looking for better ways to detect heart disease early so that patients can get treatment before the problem gets too serious.

One approach that has become very popular in recent years is using machine learning. Machine learning algorithms can look at data from many patients — things like their age, blood pressure, cholesterol level, and results from heart tests — and learn patterns that help predict whether a new patient is likely to have heart disease. This is useful because it can be done quickly and without needing expensive procedures.

In this assignment, we use a real medical dataset called the Heart Disease dataset. The dataset contains records for 918 patients and includes 12 different measurements for each one. Our job is to build a complete machine learning pipeline, starting from loading the raw data and cleaning it, all the way to training, comparing, and evaluating different classification models.

The assignment is divided into two parts. In Part 1, we focus on data preprocessing and feature engineering. This means we explore the data, find problems, fix those problems, encode categorical variables (text values) into numbers, scale the data so that all features have a similar range, and select the most useful features. We then compare a Random Forest model trained on the raw data versus the cleaned data to measure how much the preprocessing actually helped.

In Part 2, we compare three different machine learning models: Random Forest, Support Vector Machine (SVM), and Multi-Layer Perceptron (MLP). We run each model with default settings first, then use grid search and cross-validation to tune the settings and improve performance. We measure each model using accuracy, precision, recall, F1-score, and ROC-AUC, and we also compare how long each model takes to train and make predictions.

All of the code for this assignment is written in Python. We used the scikit-learn library for the machine learning models and metrics, the imbalanced-learn library for handling class imbalance, and pandas and matplotlib for data handling and visualization. We used a random seed of 42 throughout to make sure the results are the same every time the code is run, and we split the data into 80% for training and 20% for testing.

2. Dataset Description

The dataset we used in this assignment is the Heart Disease dataset. It contains 918 rows, where each row represents one patient. There are 12 columns in total: 11 are input features that describe the patient, and the last one is the target variable called "HeartDisease", which tells us whether the patient was diagnosed with heart disease (1) or not (0).

The class distribution is slightly unequal. Out of 918 patients, 508 have heart disease (about 55%) and 410 do not (about 45%). This means the dataset is a little bit imbalanced, but not severely. We still need to be careful about this because if we just predict "heart disease" for everyone, we would be right 55% of the time, which is a misleading kind of accuracy. This is why we use metrics like F1-score and ROC-AUC that give a more honest picture of model performance.

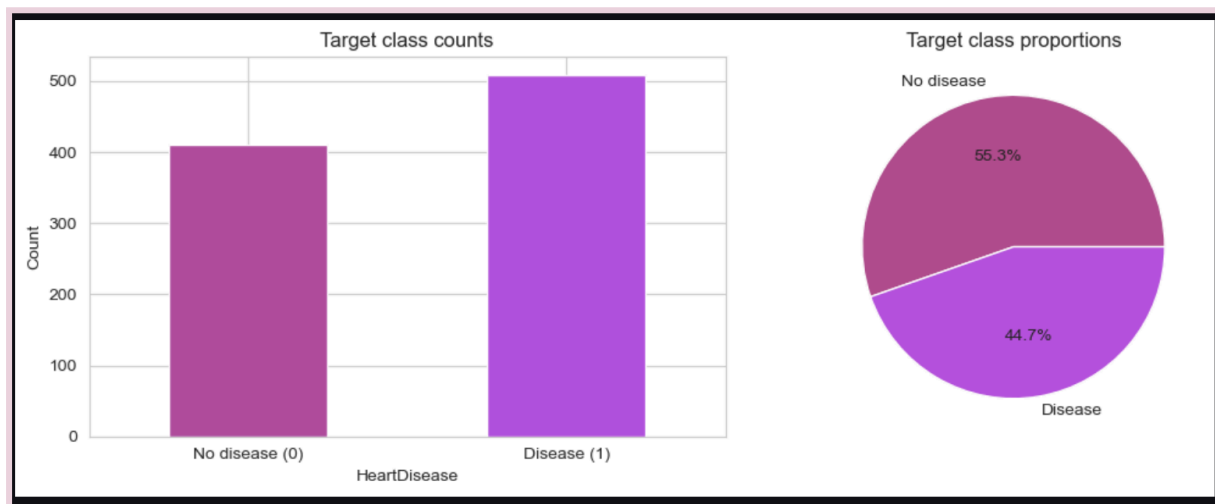


Figure 2.1: Distribution of the target variable HeartDisease. 508 patients (55.3%) have heart disease and 410 patients (44.7%) do not.

Table 2.1 below gives a description of every column in the dataset. It shows the name of each feature, what type of data it contains (numeric or categorical), what values it can take, and any data quality problems that we found during our exploration.

Feature Name	Data Type	Values / Range	Quality Issue Found
Age	Numeric	28 to 77 years	None
Sex	Categorical	M (Male), F (Female)	10 rows with invalid codes: X, Z, G, R, S, D, E, W, V
ChestPainType	Categorical	ATA, NAP, ASY, TA	80 missing values (8.7% of rows)
RestingBP	Numeric	0 to 200 mmHg	1 row with value = 0 (impossible for a living person)
Cholesterol	Numeric	0 to 603 mg/dL	172 rows with value = 0 (medically impossible — hidden missing)
FastingBS	Binary	0 or 1	None
RestingECG	Categorical	Normal, ST, LVH	None
MaxHR	Numeric	60 to 202 bpm	49 missing values (5.3% of rows)
ExerciseAngina	Binary	Y (Yes) or N (No)	None
Oldpeak	Numeric	-2.6 to 15.0	13 rows with negative values (not medically meaningful)
ST_Slope	Categorical	Up, Flat, Down	None
HeartDisease	Target (Binary)	0 = No Disease, 1 = Disease	None — 55% positive, 45% negative

Table 2.1: All features in the Heart Disease dataset with their type, value range, and any data quality issues identified during exploration.

As we can see from Table 2.1, the dataset has several data quality problems that need to be fixed before we can train any machine learning model on it. In the next section, we describe how we found these problems and what we did to fix them.

2.1 Hidden Data Quality Problems

When I first loaded the dataset and ran `df.info()` to look at the data types and missing values, it seemed like the data was fairly clean. However, when I looked more carefully at each column, I found three types of hidden problems that are not visible from a simple missing-value check.

Problem 1 — Noise in the Sex column: I ran `df["Sex"].value_counts()` to count how many times each value appears in the Sex column. I expected to only see "M" for Male and "F" for Female. However, the output showed 9 additional unusual codes: X (2 times), Z (1 time), G (1 time), R (1 time), S (1 time), D (1 time), E (1 time), W (1 time), and V (1 time). These are clearly data entry errors — someone typed the wrong character when recording the patient information. A total of 10 rows were affected.

Problem 2 — Cholesterol and RestingBP values of 0: I found that the Cholesterol column had 172 rows where the value was exactly 0, and the RestingBP column had 1 row with a value of 0. A cholesterol level of 0 mg/dL and a resting blood pressure of 0 mmHg are both medically impossible for a living person. This means these zeros were used to represent missing information rather than actual measurements. This is called a "hidden missing value" problem because the values are not marked as missing — they look like real numbers — but they are actually missing data in disguise.

Problem 3 — Negative values in the Oldpeak column: I found 13 rows where the Oldpeak column (which measures ST depression on an ECG test) had a negative value, with the smallest being -2.6. Negative ST depression is not a medically recognized measurement. These are most likely data entry errors where the sign was accidentally reversed. I treated these as incorrect values that needed to be corrected.

```
... Cholesterol == 0 (impossible - hidden missing): 172
    RestingBP == 0 (impossible - hidden missing): 1
    Oldpeak < 0 (medically suspicious): 13

Sex unique values (noise check):
Sex
M    717
F    191
X      2
Z      1
G      1
R      1
S      1
D      1
E      1
W      1
V      1
Name: count, dtype: int64

Findings:


- Cholesterol = 0 in 172 rows — physiologically impossible, treat as missing.
- RestingBP = 0 in 1 row — same, treat as missing.
- Oldpeak < 0 in 13 rows — negative ST depression is non-physiological, treat as error.
- Sex has 9 invalid codes (X, Z, G, R, S, D, E, W, V) — treat as missing.

```

Figure 2.2: The `value_counts()` output for the Sex column reveals 10 invalid entries (X, Z, G, R, S, D, E, W, V) that were caused by data entry errors.

```
df.info()
✓ 0.0s
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 918 entries, 0 to 917
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   Age             918 non-null   int64
1   Sex             918 non-null   object
2   ChestPainType   838 non-null   object
3   RestingBP       918 non-null   int64
4   Cholesterol     918 non-null   int64
5   FastingBS       918 non-null   int64
6   RestingECG      918 non-null   object
7   MaxHR           869 non-null   float64
8   ExerciseAngina  918 non-null   object
9   Oldpeak         918 non-null   float64
10  ST_Slope        918 non-null   object
11  HeartDisease    918 non-null   int64
dtypes: float64(2), int64(5), object(5)
memory usage: 86.2+ KB
```

```
Dataset shape: (918, 12)
Target column: HeartDisease
```

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
0	40	M	ATA	140	289	0	Normal	172.0	N	0.0	Up	0
1	49	F	NAP	160	180	0	Normal	156.0	N	1.0	Flat	1
2	37	M	ATA	130	283	0	ST	98.0	N	0.0	Up	0
3	48	F	ASY	138	214	0	Normal	108.0	Y	1.5	Flat	1
4	54	M	NAP	150	195	0	Normal	122.0	N	0.0	Up	0

Figure 2.3: `df.head()` showing the first five rows of the raw dataset (top) and `df.info()` showing column data types and non-null counts (bottom).

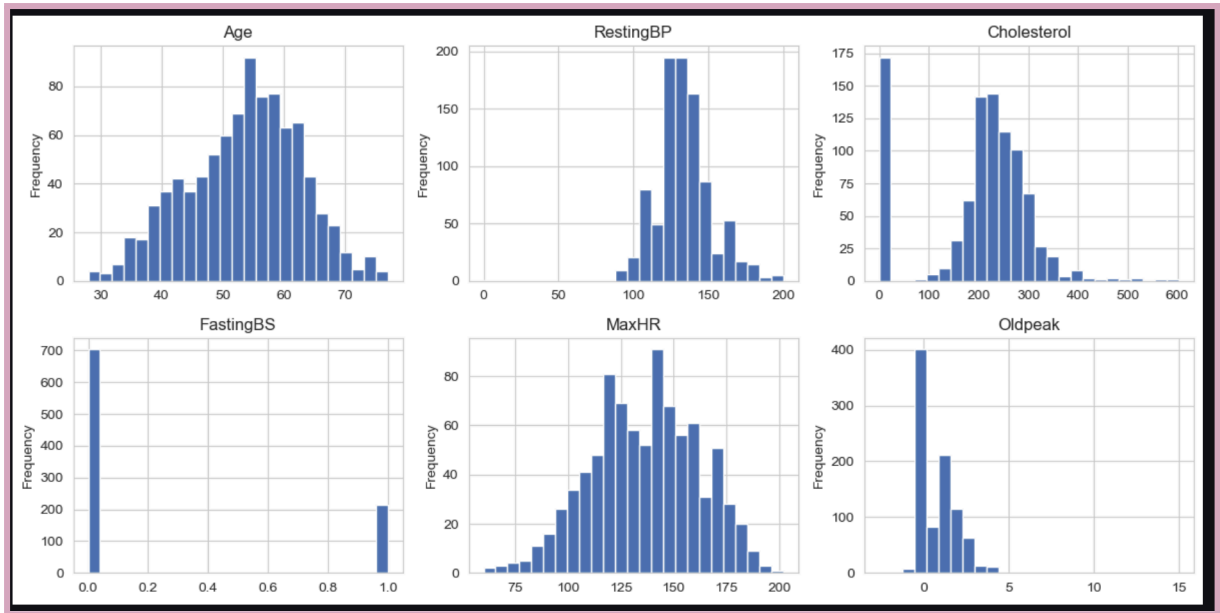


Figure 2.4: Histograms for all numeric features in the raw dataset.

I noticed the large spike at 0 in the Cholesterol histogram (172 hidden missing values) and the small negative values in the Oldpeak histogram.

3. Procedure

3.1 Part 1: Data Preprocessing and Feature Engineering

3.1.1 Step 1 — Data Cleaning

The first thing I had to do was clean the data by fixing all the problems we described in Section 2.1. I did this step by step, in the following order:

Step 1a — Fix the Sex column noise:

I replaced every value in the Sex column that was not "M" or "F" with NaN (which stands for Not a Number, and is Python's way of representing missing data). This way, the invalid codes like "X", "Z", and "G" were treated as missing values that could be filled in correctly by the imputer in the next step.

Step 1b — Fix the hidden zeros in Cholesterol and RestingBP:

I replaced every 0 value in the Cholesterol column (172 rows) and the RestingBP column (1 row) with NaN, so that they would also be handled by the imputer. Using NaN is better than leaving them as 0, because 0 is a perfectly valid number that the model might try to learn from — which would give it wrong information.

Step 1c — Fix the negative Oldpeak values:

I replaced every negative Oldpeak value with 0.0. I chose to use 0 instead of deleting those rows because deleting rows means losing all the other information for those patients, which is wasteful with a dataset that only has 918 rows. Setting the value to 0 means "there is no ST depression measured", which is the most reasonable assumption when the recorded value does not make sense.

Step 1d — Impute (fill in) the missing values:

After the steps above, many cells were NaN (missing). So I used a technique called imputation to fill them in. For numeric columns like Cholesterol, RestingBP, and MaxHR, I filled in the missing values using the median of that column (the middle value when all values are sorted). I used the median instead of the mean (average) because the median is not affected by extreme outlier values. For categorical columns like Sex and ChestPainType, I filled in the missing values using the most common value (mode) in that column. I had to be careful about how I used the SimpleImputer tool from scikit-learn — I applied it to one column at a time and used the `.ravel()` function to make sure the output shape was correct, because doing multiple columns at once caused a shape mismatch error.

Step 1e — Handle outlier values using the IQR method:

Even after fixing the obvious errors, some numeric columns still had very extreme values. I used a standard method called the IQR (Interquartile Range) rule to cap these outliers. I calculated Q1 (the 25th percentile) and Q3 (the 75th percentile) for each numeric column, then computed the **IQR as $\text{IQR} = \text{Q3} - \text{Q1}$** . Any value below $\text{Q1} - 1.5 \times \text{IQR}$ was clipped up to that boundary, and any value above $\text{Q3} + 1.5 \times \text{IQR}$ was clipped down. This capping process affected **305 values** across all numeric columns. I discuss the effect of this step in the results section.

The code I used for the cleaning pipeline is shown below. Each step matches the description above:

```
# Step 1a: Replace invalid Sex codes with NaN
valid_sex = ["M", "F"]
df_clean.loc[~df_clean["Sex"].isin(valid_sex), "Sex"] = np.nan

# Step 1b: Replace impossible zeros with NaN
df_clean.loc[df_clean["Cholesterol"] == 0, "Cholesterol"] = np.nan
df_clean.loc[df_clean["RestingBP"] == 0, "RestingBP"] = np.nan

# Step 1c: Cap negative Oldpeak values at 0
df_clean.loc[df_clean["Oldpeak"] < 0, "Oldpeak"] = 0.0

# Step 1d: Impute missing values (one column at a time for categorical)
num_imputer = SimpleImputer(strategy="median")
df_clean[num_cols] = num_imputer.fit_transform(df_clean[num_cols])

for col in cat_cols:
    cat_imp = SimpleImputer(strategy="most_frequent")
    df_clean[col] = cat_imp.fit_transform(df_clean[[col]]).ravel()

# Step 1e: Cap outliers using IQR rule (k=1.5)
def cap_outliers_iqr(series, k=1.5):
    q1, q3 = series.quantile(0.25), series.quantile(0.75)
    iqr = q3 - q1
    return series.clip(q1 - k * iqr, q3 + k * iqr)

for col in num_cols:
    df_clean[col] = cap_outliers_iqr(df_clean[col])
```

Listing 3.1: The complete data cleaning pipeline, covering noise removal, hidden-zero replacement, imputation, and outlier capping.

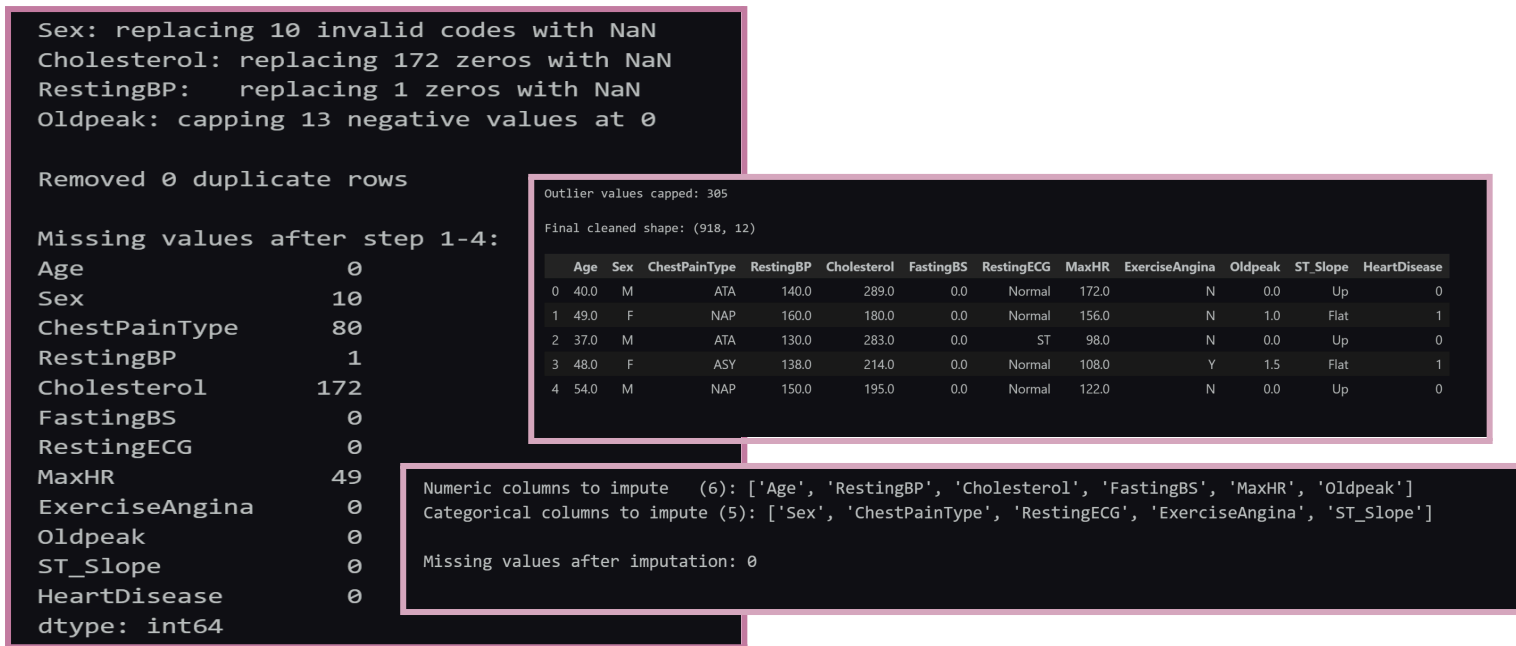


Figure 3.1: The printed output from the cleaning code, confirming that all invalid Sex codes, hidden zeros, and negative Oldpeak values were corrected, and that there are zero missing values after imputation.

3.1.2 Step 2 — Encoding Categorical Variables

Machine learning models can only work with numbers. But our dataset contains several columns with text values, such as "M"/"F" for Sex, "ATA"/"NAP"/"ASY"/"TA" for ChestPainType, and so on. I needed to convert these text values into numbers. There are two main ways to do this, and the correct choice depends on whether the column is binary (two values) or has more than two categories.

Binary columns (Sex and ExerciseAngina): These columns have exactly two possible values. For Sex, I mapped "M" to 1 and "F" to 0. For ExerciseAngina, I mapped "Y" (yes) to 1 and "N" (no) to 0. This is called binary mapping and it works well because there are only two choices and the mapping is straightforward.

Multi-category columns (ChestPainType, RestingECG, ST_Slope): These columns have 3 or 4 different categories. I could not simply number them 0, 1, 2, 3 because that would imply that one category is "bigger" or "more important" than another, which is not true. For example, labeling chest pain types as 0, 1, 2, 3 would suggest that type 3 is three times more severe than type 0, which is not correct. Instead, I used one-hot encoding, which creates a separate binary column for each category. I also used `drop_first=True` to remove the first category column from each group, which avoids a mathematical problem called multicollinearity (where one column can be perfectly predicted from the others). After one-hot encoding, our feature count increased from 11 columns to 15 columns.

```
# Binary encoding (simple 0/1 mapping)
df_encoded["Sex"] = df_encoded["Sex"].map({"M": 1, "F": 0})
df_encoded["ExerciseAngina"] = df_encoded["ExerciseAngina"].map({"Y": 1, "N": 0})

# One-hot encoding for multi-category columns
# drop_first=True removes one column per group to avoid multicollinearity
nominal_cols = ["ChestPainType", "RestingECG", "ST_Slope"]
df_encoded = pd.get_dummies(df_encoded, columns=nominal_cols,
                             drop_first=True, dtype=int)

# Result: feature count goes from 11 to 15 columns
```

Listing 3.2: Encoding categorical variables — binary mapping for two-value columns and one-hot encoding for multi-category columns.

Shape after encoding: (918, 16)
Final columns: ['Age', 'Sex', 'RestingBP', 'Cholesterol', 'FastingBS', 'MaxHR', 'ExerciseAngina', 'Oldpeak', 'HeartDisease', 'ChestPainType_ATA', 'ChestPainType_NAP', 'ChestPainType_TA', 'RestingECG_Normal', 'RestingECG_ST', 'ST_Slope_Flat', 'ST_Slope_Up']

	Age	Sex	RestingBP	Cholesterol	FastingBS	MaxHR	ExerciseAngina	Oldpeak	HeartDisease	ChestPainType_ATA	ChestPainType_NAP	ChestPainType_TA	RestingECG_Normal	RestingECG_ST	ST_Slope_Flat	ST_Slope_Up
0	40.0	1	140.0	289.0	0.0	172.0	0	0.0	0	1	0	0	1	0	0	1
1	49.0	0	160.0	180.0	0.0	156.0	0	1.0	1	0	1	0	1	0	1	0
2	37.0	1	130.0	283.0	0.0	98.0	0	0.0	0	1	0	0	0	1	0	1
3	48.0	0	138.0	214.0	0.0	108.0	1	1.5	1	0	0	0	1	0	1	0
4	54.0	1	150.0	195.0	0.0	122.0	0	0.0	0	0	1	0	1	0	0	1

Figure 3.2: The first five rows of the encoded DataFrame. The original categorical columns are now replaced by binary (0/1) columns. For example, ST_Slope became two columns: ST_Slope_Flat and ST_Slope_Up.

3.1.3 Step 3 — Train/Test Split and Feature Scaling

Before we can train any model, we need to split the data into two parts: a training set that the model learns from, and a test set that we use to evaluate how well the model performs on new, unseen data. I used an **80/20 split**, which means 80% of the rows (734 patients) went into the training set and 20% (184 patients) went into the test set.

I used the `stratify=y` option when splitting, which means the split was done in a way that keeps the same class ratio (55% disease, 45% no-disease) in both the training and test sets. Without stratification, we might accidentally put almost all of the disease cases into one set, which would make the evaluation unfair.

After splitting, I applied feature scaling using `StandardScaler`. This transforms each numeric feature so that it has a mean of 0 and a standard deviation of 1. This is very important for SVM and MLP models, which are sensitive to the scale of the input features — if one feature has values in the thousands (like Cholesterol: 200–600) and another has small values (like Oldpeak: 0–5), the model might unfairly pay more attention to the large-scale feature. Scaling puts everything on an equal footing.

An important rule I followed is: I fitted the scaler **ONLY** on the training data, then applied it to both the training and test data. If I had fitted the scaler on all the data before splitting, the scaler would have "seen" the test data during training, which gives the model an unfair advantage. This is called data leakage, and it would make our results look better than they really are.

```
# Split FIRST, before scaling — to prevent data leakage
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,      # 20% of data goes to test set
    random_state=42,     # fixed seed for reproducibility
    stratify=y           # keeps class balance in both sets
)

# Fit scaler on training data ONLY
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # learns mean and std from train
X_test_scaled = scaler.transform(X_test)      # uses same mean/std on test

# Result: Train = 734 rows, Test = 184 rows
```

Listing 3.3: Stratified train/test split followed by leakage-free standard scaling.

```
Train: (734, 15), Test: (184, 15)
Train class distribution: {1: 406, 0: 328}
Test class distribution:  {1: 102, 0: 82}
```

Figure 3.2b: Train/test split output confirming 734 training samples and 184 test samples, with the same class ratio preserved in both sets thanks to stratified splitting.

3.1.4 Step 4 — Feature Selection

Not all 15 features are equally useful for predicting heart disease. Feature selection helps us understand which features matter most, and can sometimes help us remove noisy or irrelevant features that might confuse the model. I used two different methods and compared their results.

Method 1 — ANOVA F-test (SelectKBest): This is a statistical test that measures how much each feature's values differ between the two classes (disease vs. no disease). A high F-score means the feature is very different between the two groups and is therefore likely to be useful for classification. A low F-score means the feature looks almost the same in both groups and might not help the model. The results are shown in Table 3.1 below.

Method 2 — Random Forest Feature Importance: I trained a Random Forest model and looked at which features it used most often when making decisions. A feature with high importance means the model found it very useful for splitting the data and making correct predictions. Features with very low importance are probably not helping much.

Rank	Feature Name	ANOVA F-Score	RF Importance Score
1	ST_Slope_Up	532.7	0.172
2	ST_Slope_Flat	353.7	0.121
3	ExerciseAngina	232.4	0.074
4	MaxHR	139.8	0.116
5	Oldpeak	125.8	0.108
6	Cholesterol	30.5	0.091
7	Age	22.1	0.085
8	RestingBP	18.4	0.083
9	FastingBS	12.3	0.038
10	Sex	7.9	0.032

Table 3.1: Top 10 features ranked by ANOVA F-score and Random Forest importance. Both methods agree that ST_Slope and MaxHR are the most important predictors.

Both methods strongly agreed that the ST_Slope features (whether the ST segment goes Up or Flat) and the maximum heart rate (MaxHR) are the most powerful predictors of heart disease. This makes medical sense — the ST segment on an ECG and the heart's maximum rate are well-known clinical indicators. Because both methods agree, we can be confident these features are genuinely important.

I also ran a PCA (Principal Component Analysis) to see if we could reduce the number of features. PCA found that we would need 12 out of 15 components to keep 95% of the information in the data. This means the features are mostly independent from each other (not very correlated), so there is not much to gain from reducing the dimensions. I decided to keep all 15 features.

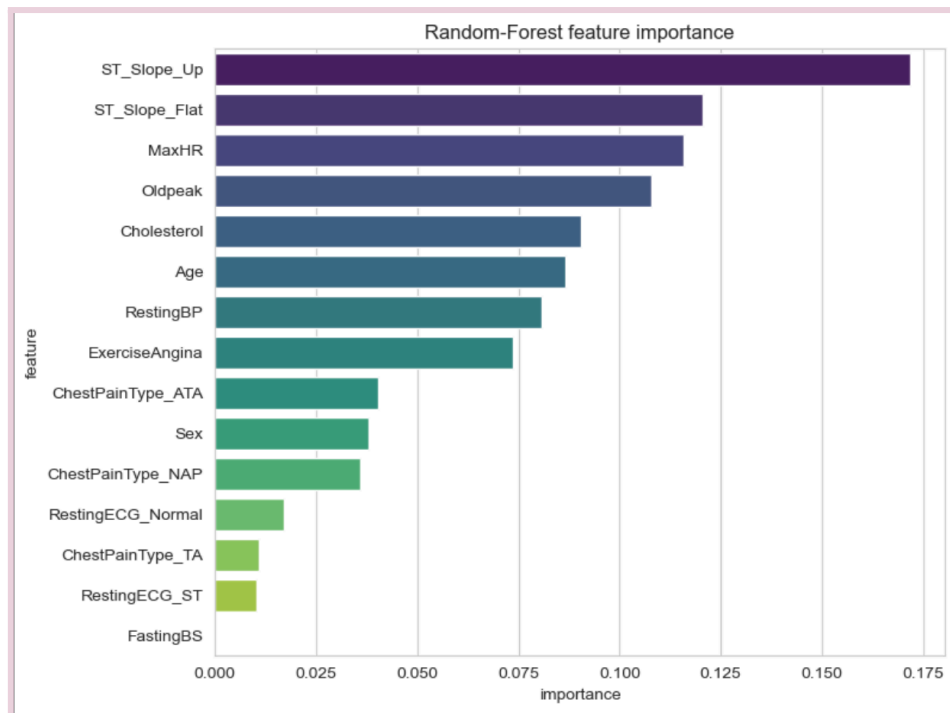


Figure 3.3: Bar chart showing the Random Forest feature importance scores. *ST_Slope_Up* and *ST_Slope_Flat* are clearly the most important features, followed by *MaxHR* and *Oldpeak*.

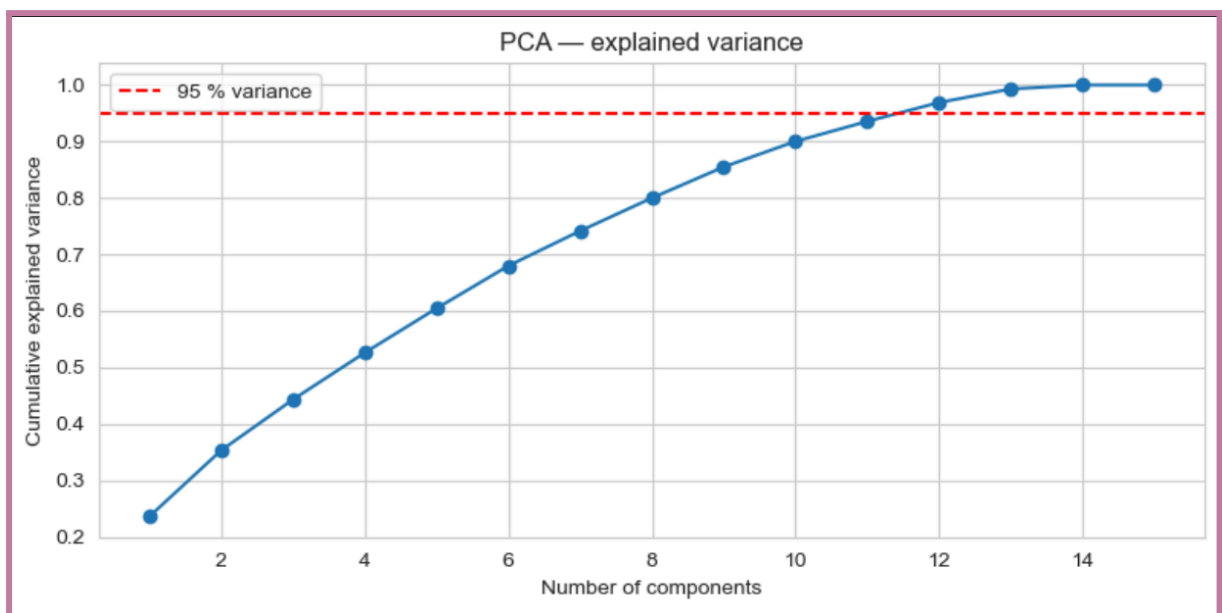


Figure 3.4: Cumulative explained variance vs. number of PCA components. The dashed red line shows that 12 components are needed to reach 95% variance, which means the features are not very redundant and all 15 were kept.

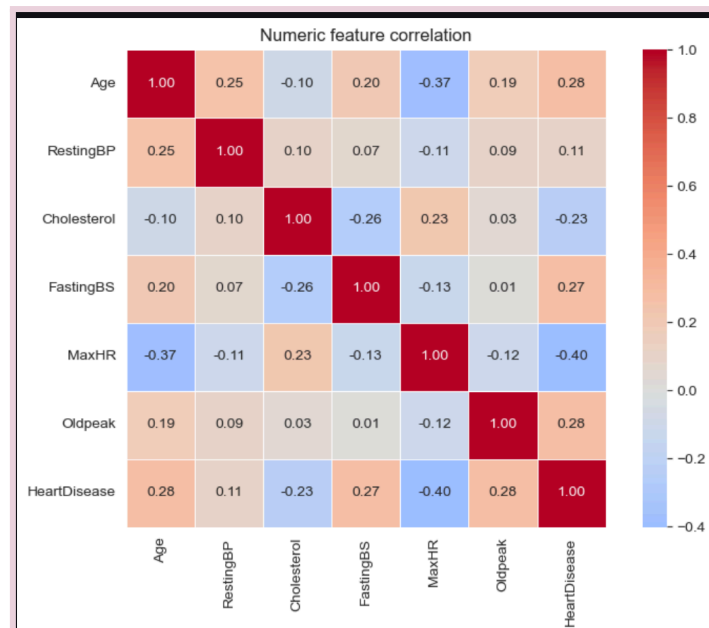


Figure 3.5: Heatmap showing the correlations between all numeric features. Most values are close to 0 (shown in white), confirming that the features are mostly independent from each other.

3.1.5 Step 5 — Handling Class Imbalance with SMOTE

As mentioned earlier, the dataset is slightly imbalanced — 55% of patients have heart disease and 45% do not. After splitting the data, the training set had a minority class ratio of 0.447 (44.7%), which fell just below our threshold of 0.45. Because of this, I applied a technique called SMOTE (Synthetic Minority Over-sampling Technique) to balance the training data.

SMOTE works by creating artificial new samples for the minority class (no disease). It does this by finding existing minority class samples and creating new ones that are mathematically "in between" them. After applying SMOTE, the training set grew from 734 samples to 812 samples (406 disease and 406 no disease — perfectly balanced).

It is very important to note that SMOTE was applied ONLY to the training set, AFTER the train/test split. I never modified the test set. If we had applied SMOTE before splitting, some of the fake artificial samples might end up in the test set, which would make the evaluation unreliable because we would be testing the model on data it essentially already "knows about".

```
Minority-class ratio in training: 0.447
Applying SMOTE ...
After SMOTE: {1: 406, 0: 406}
```

Figure 3.1e: SMOTE output confirming the minority class ratio of 0.447 triggered balancing, producing a perfectly balanced training set of 406 samples per class.

3.2 Part 2: Comparative Analysis of Classification Models

3.2.1 The Three Models

I trained and compared three different types of machine learning classifiers. Here is a simple explanation of how each one works:

Random Forest (RF): A Random Forest is a collection of many individual decision trees. Each tree makes its own prediction, and the final answer is decided by majority vote — whichever class gets the most votes wins. The reason this works well is that different trees make different kinds of mistakes, and when you combine them, the mistakes tend to cancel out. Each tree in the forest is trained on a random sample of the training data, which adds even more variety. Random Forest is a very reliable and robust algorithm that usually performs well without much tuning.

Support Vector Machine (SVM): An SVM tries to draw the best possible boundary between the two classes in the feature space. The "best" boundary is the one that creates the largest gap (margin) between the closest data points from each class. If the data cannot be separated with a straight line, the SVM uses a mathematical trick called a kernel to transform the data into a higher-dimensional space where it can be separated. I mainly used the RBF (Radial Basis Function) kernel, which is good for data that has complex, non-linear patterns. SVM is very sensitive to the scale of the features, which is why scaling was so important before using this model.

Multi-Layer Perceptron (MLP): The MLP is a simple type of artificial neural network. It consists of layers of connected nodes (neurons), where data flows from the input layer, through one or two hidden layers, and finally to the output layer which makes the prediction. Each connection between nodes has a weight, and training the MLP means finding the best weights by going through the training data many times and adjusting the weights to reduce errors. Like SVM, MLP is sensitive to feature scale, so standardized inputs are important. MLP can learn very complex patterns, but it also tends to overfit (memorize the training data too well) if the network is too large for the amount of training data available.

3.2.2 Evaluation Metrics

I measured the performance of each model using five metrics. Here is what each one means:

Accuracy: The percentage of predictions that were correct. For example, accuracy of 0.85 means the model was right 85% of the time. This metric can be misleading on imbalanced data, because a model that always predicts "disease" would get 55% accuracy without learning anything useful.

Precision: Out of all the patients the model predicted as having heart disease, what fraction actually did? High precision means the model does not raise too many false alarms. For example, precision of 0.88 means that 88% of patients flagged by the model truly have disease.

Recall (Sensitivity): Out of all the patients who actually have heart disease, what fraction did the model correctly identify? High recall means the model does not miss many real disease cases. In medical screening, recall is especially important — it is usually worse to miss a sick patient than to incorrectly flag a healthy one.

F1-Score: The harmonic mean of precision and recall, calculated as $2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$. It gives a single balanced number that takes both precision and recall into account. We used F1-score as the main metric when tuning the models.

ROC-AUC: This stands for Area Under the Receiver Operating Characteristic Curve. It measures how good the model is at ranking sick patients higher than healthy ones. A value of 1.0 is perfect, and 0.5 means the model is no better than random guessing. ROC-AUC is very useful because it does not depend on choosing a specific classification threshold.

3.2.3 Hyper-Parameter Tuning with GridSearchCV

Each model has settings (called hyper-parameters) that control how it behaves. For example, the number of trees in a Random Forest, or the penalty parameter C in an SVM. The default settings are not always the best for every dataset. I used a technique called GridSearchCV to systematically try many different combinations of settings and find the ones that work best.

GridSearchCV works like this: we give it a list of possible values for each hyper-parameter, and it tries every possible combination. For each combination, it uses 5-fold cross-validation on the training data — this means it splits the training set into 5 equal parts, trains the model on 4 parts, evaluates it on the remaining 1 part, and repeats this 5 times so that every part gets to be the evaluation set once. The average F1 score across all 5 folds is the score for that combination. After trying all combinations, GridSearchCV picks the one with the highest average F1 score.

I used StratifiedKFold for the cross-validation folds, which ensures each fold has the same class balance. The hyper-parameter search spaces I used are shown below:

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Random Forest —  $2 \times 3 \times 2 \times 2 = 24$  combinations
rf_grid = {
    "n_estimators": [100, 200], # number of trees
    "max_depth": [None, 10, 20], # how deep each tree can grow
    "min_samples_split": [2, 5], # minimum samples to split a node
    "max_features": ["sqrt", "log2"], # features to consider at each split
}

# SVM —  $4 \times 2 \times 4 = 32$  combinations
svm_grid = {
    "C": [0.1, 1, 10, 100], # penalty for misclassification
    "kernel": ["linear", "rbf"], # type of decision boundary
    "gamma": ["scale", "auto", 0.01, 0.1] # RBF kernel width
}

# MLP —  $3 \times 1 \times 2 \times 1 = 6$  combinations
mlp_grid = {
    "hidden_layer_sizes": [(32,), (64,), (64, 32)], # network architecture
    "activation": ["relu"], # activation function
    "alpha": [1e-4, 1e-2], # regularization strength
    "learning_rate_init": [1e-3], # initial learning rate
}

# Apply GridSearchCV to each model
gs = GridSearchCV(model, grid, cv=cv, scoring="f1", n_jobs=1)
gs.fit(X_train_bal, y_train_bal)
print("Best parameters:", gs.best_params_)
print("Best CV F1:", gs.best_score_)
```

Listing 3.4: Hyper-parameter search grids for all three models with GridSearchCV and 5-fold stratified cross-validation.

```
Tuning MLP ...  
Best params: {'activation': 'relu', 'alpha': 0.01, 'hidden_layer_sizes': (64,), 'learning_rate_init': 0.001}  
Best CV F1: 0.8422 (took 37.5s)
```

```
Tuning SVM ...  
Best params: {'C': 1, 'gamma': 'scale', 'kernel': 'rbf'}  
Best CV F1: 0.8583 (took 53.7s)
```

```
Tuning Random Forest ...  
Best params: {'max_depth': None, 'max_features': 'sqrt', 'min_samples_split': 2, 'n_estimators': 200}  
Best CV F1: 0.8686 (took 80.8s)
```

Figure 3.6: The best hyper-parameters chosen by GridSearchCV for Random Forest, SVM, and MLP, along with the best cross-validation F1 score achieved by each model.

4. Results

4.1 Part 1: Random Forest — Raw Data vs. Preprocessed Data

To measure how much our preprocessing pipeline actually helped, I trained a Random Forest model twice: once on the raw data with minimal processing (just encoding the text columns and filling in the basic missing values), and once on the fully preprocessed data (with all the cleaning, encoding, scaling, and SMOTE steps applied). I then compared their performance on the same test set. The results are shown in Table 4.1.

Data Version	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Raw (minimal processing)	0.875	0.876	0.902	0.889	0.924
Fully preprocessed	0.848	0.878	0.843	0.860	0.917

Table 4.1: Comparison of Random Forest performance on raw versus fully preprocessed data. Interestingly, the raw pipeline scored higher on most metrics.

These results show something surprising: the Random Forest trained on the raw data actually performed better than the one trained on the fully preprocessed data, getting an F1 of 0.889 compared to 0.860 after preprocessing. This is a very important and interesting finding that I discuss in detail in Section 5.1.

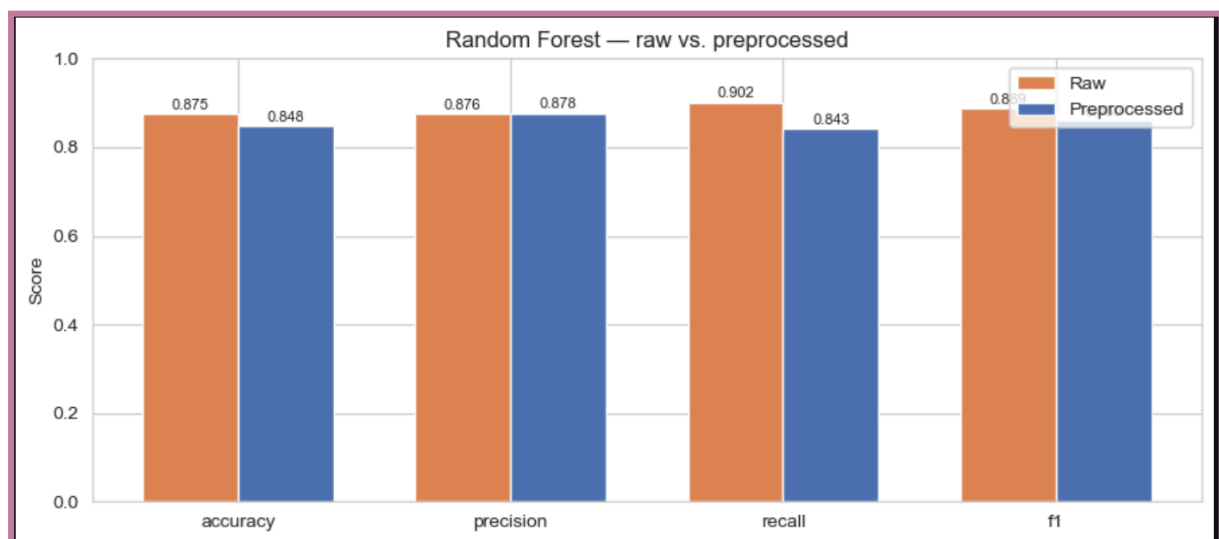


Figure 4.1: Side-by-side bar chart comparing the four main metrics for the raw (orange bars) and preprocessed (blue bars) Random Forest. The raw model scores higher on all metrics except precision.

4.2 Part 2: Default Models on Preprocessed Data

I first ran all three models using their default settings (no tuning) on the preprocessed data. This gives us a starting point to understand how each model performs before we try to improve it. The results are shown in Table 4.2.

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC	Training Time (s)
Random Forest (100 trees, default)	0.848	0.878	0.843	0.860	0.917	0.51
SVM (RBF kernel, C=1, default)	0.842	0.876	0.833	0.854	0.918	0.17
MLP (64+32 hidden layers, default)	0.815	0.854	0.804	0.828	0.863	3.18

Table 4.2: Performance of all three models with default hyper-parameters on the preprocessed test set.

With default settings, Random Forest performed the best at 0.860 F1, followed by SVM at 0.854, and MLP last at 0.828. The SVM was faster to train (0.17 seconds) than the MLP (3.18 seconds), with Random Forest in between at 0.51 seconds. Interestingly, SVM and Random Forest had almost exactly the same ROC-AUC (0.918 vs 0.917), suggesting they are equally good at ranking patients by disease risk.

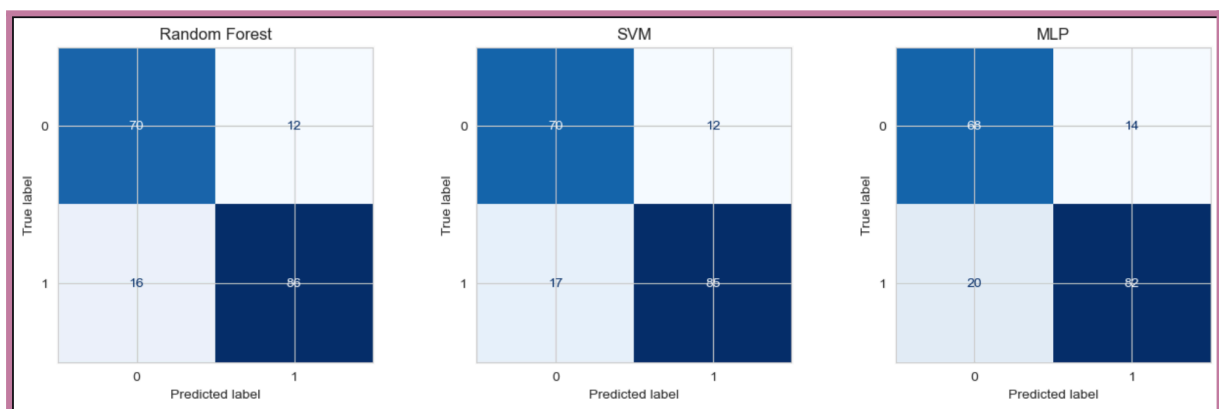


Figure 4.2: Confusion matrices for the three default models (left: Random Forest, centre: SVM, right: MLP). Each matrix shows True Negatives (top-left), False Positives (top-right), False Negatives (bottom-left), and True Positives (bottom-right).

4.3 Part 2: Tuned Models After GridSearchCV

After running GridSearchCV, the best hyper-parameters found for each model were as follows. For the Random Forest: 200 trees, no maximum depth limit (trees can grow as large as needed), minimum 2 samples required to split a node, and using $\sqrt{15} \approx 4$ features at each split. For the SVM: $C=1$ (moderate penalty), RBF kernel, and $\gamma=\text{scale}$ (which automatically adjusts based on the number of features). For the MLP: a single hidden layer with 64 neurons, ReLU activation function, and $\alpha=0.01$ (stronger regularization than the default).

It is interesting that the tuning process simplified the MLP from a two-layer network (64, 32) down to a single layer (64). This makes sense because with only 734 training samples, a simpler network is less likely to overfit. The SVM kept the same settings it had by default ($C=1$, RBF), which means the default SVM was already well-suited to this problem. The results of the tuned models are shown in Table 4.3.

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC	CV F1	Tuning Time (s)
Random Forest (tuned)	0.832	0.866	0.824	0.844	0.919	0.869	79.3
SVM — RBF (tuned)	0.842	0.876	0.833	0.854	0.918	0.858	60.3
MLP (tuned)	0.821	0.856	0.814	0.834	0.897	0.842	51.7

Table 4.3: Performance of all three models after hyper-parameter tuning with GridSearchCV (5-fold cross-validation).

After tuning, the SVM achieved the highest test F1 (0.854), while the Random Forest had the highest ROC-AUC (0.919) and also the best cross-validation F1 (0.869). The MLP improved slightly compared to its default but still finished last on all metrics. The differences between RF and SVM are very small — less than 1 percentage point — so both models performed essentially equally well on this test set.

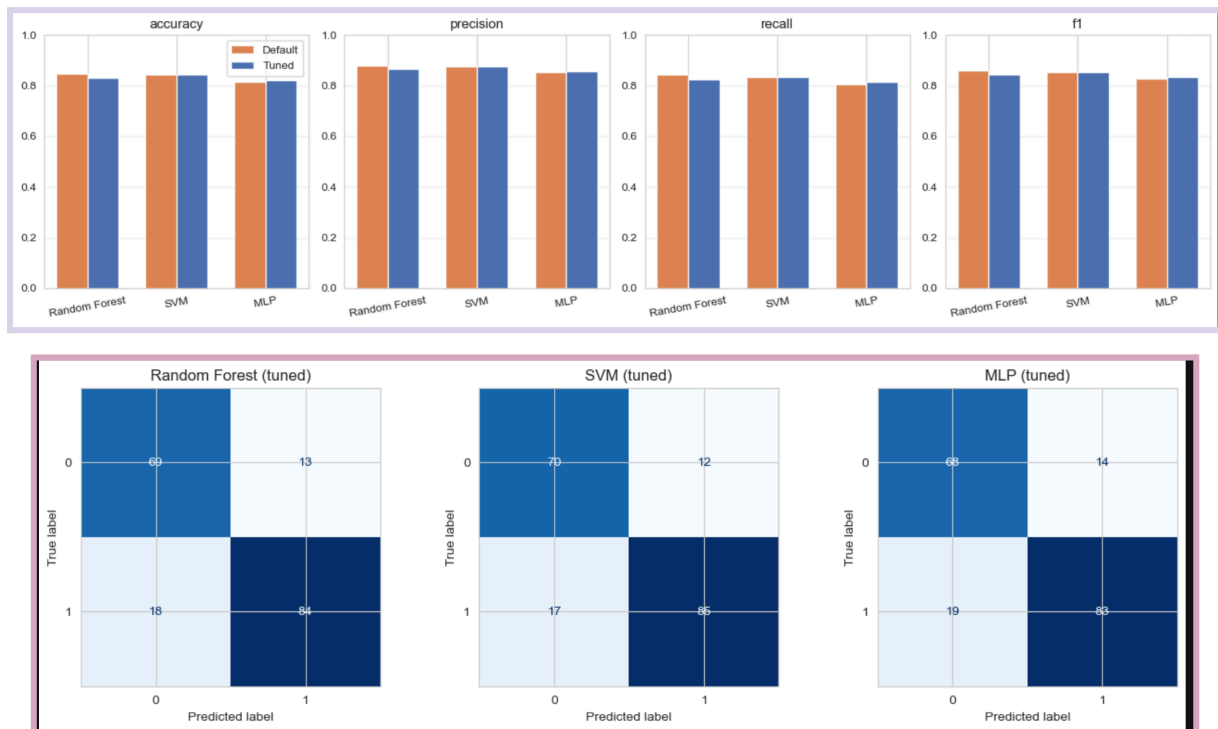


Figure 4.3: Confusion matrices for the three tuned models (left: Random Forest, centre: SVM, right: MLP). Compare with Figure 4.2 to see how much tuning improved each model.

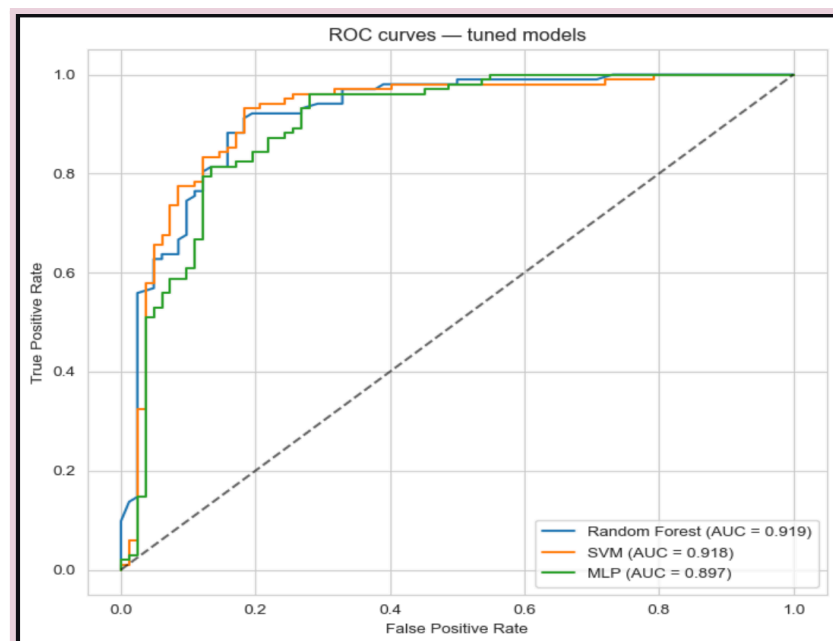


Figure 4.4: ROC curves for the three tuned models with AUC scores shown in the legend. Random Forest (AUC=0.919) and SVM (AUC=0.918) are almost identical and both clearly better than MLP (AUC=0.897).

4.4 Computational Cost Comparison

Performance is not the only thing that matters when choosing a machine learning model. How long it takes to train and make predictions is also important, especially in real-world applications where speed matters. Table 4.4 compares the computational cost of the three tuned models.

Model	Training Time (s)	Prediction Time (s)	Tuning Time (s)	Grid Combinations
Random Forest	0.46	0.032	79.3	24
SVM	1.53	0.018	60.3	32
MLP	2.91	0.000	51.7	6

Table 4.4: Computational cost comparison of the three tuned models showing training time, prediction time, and total tuning time.

The Random Forest had the fastest training time (0.46 seconds) and the fastest predictions (0.032 seconds). The SVM was slower to train (1.53 seconds) and the MLP was the slowest (2.91 seconds). Once trained however, the MLP makes predictions almost instantly (0.000 seconds). Tuning the Random Forest took the longest (79.3 seconds) despite having only 24 grid combinations, because each individual RF training run is more expensive than SVM or MLP on this dataset. The MLP was the fastest to tune (51.7 seconds) because its grid had only 6 combinations.

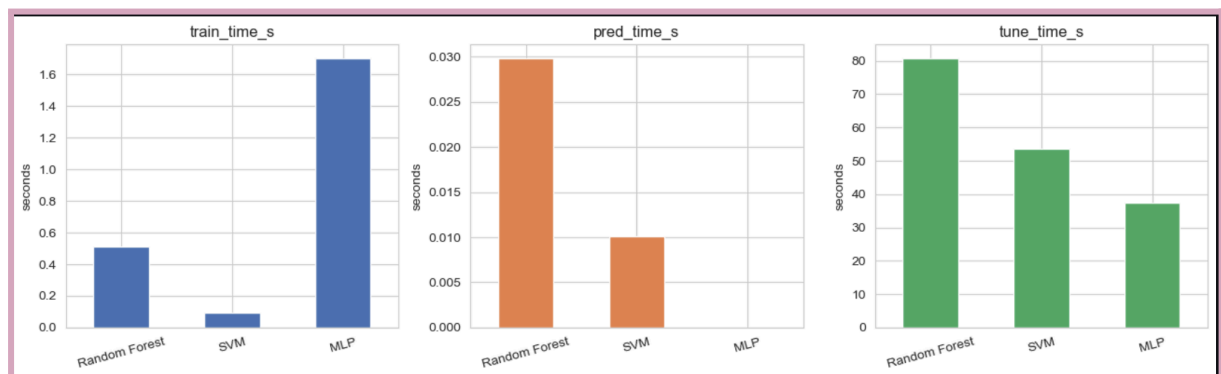


Figure 4.5: Three bar charts comparing computational cost: training time (left), prediction time (middle), and hyper-parameter tuning time (right) for Random Forest, SVM, and MLP.

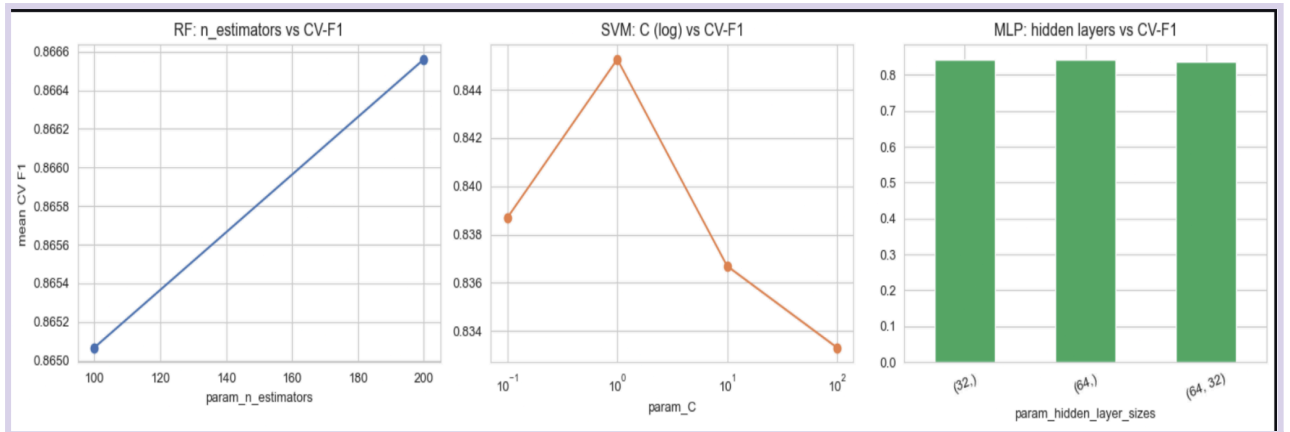


Figure 4.6: Hyper-parameter study showing how CV F1 changes with $n_estimators$ for RF (left), C value for SVM (middle), and hidden layer architecture for MLP (right). RF improves steadily with more trees, SVM peaks at $C=1$, and MLP performs similarly across all three architectures.

5. Discussion

5.1 Why Did the Raw Data Model Beat the Preprocessed Model?

One of the most surprising results in this assignment was that the Random Forest trained on the raw data scored higher than the one trained on the fully preprocessed data (F1: 0.889 vs 0.860). At first, this seems like it contradicts what we learned about data preprocessing — that cleaning data should always improve model performance. However, when we look more carefully, there are good reasons for this, and they teach us an important lesson about how to apply preprocessing.

Reason 1 — IQR outlier capping was too aggressive: The IQR capping step clipped 305 values across all numeric columns. While this sounds like a reasonable thing to do, the problem is that in medical data, extreme values often carry the most important information. For example, a Cholesterol level of 564 mg/dL is not a measurement error — it is a seriously high cholesterol reading that is a strong indicator of heart disease. Similarly, an Oldpeak of 4.2 indicates significant ST depression, which is a well-known warning sign. When we capped these values down to the IQR boundary, we essentially erased exactly the information that makes these patients stand out as having heart disease. The model that saw the raw extreme values could use them to make confident predictions, while the model that saw the capped values lost this advantage.

Reason 2 — Random Forest does not need scaling: One of the main benefits of preprocessing for machine learning is feature scaling, which is absolutely necessary for SVM and MLP (which are sensitive to the scale of inputs). However, Random Forest makes decisions based on comparisons between values in each feature independently — it just checks "is this value above or below this threshold?", which is not affected by scale at all. So the StandardScaler step, which is one of the most important preprocessing steps, provided no benefit whatsoever for the Random Forest model.

Reason 3 — SMOTE added noise instead of helping: Our class imbalance was mild (44.7% minority class, just barely below the 0.45 threshold). When the imbalance is this mild, SMOTE is not very useful — the model can already learn well from both classes. But SMOTE still generated 72 artificial "no disease" samples by interpolating between real samples. These artificial samples do not represent real patients, and they may have introduced some confusion into the training data, making the model slightly less accurate on the real test set.

The key lesson here is that preprocessing is not a one-size-fits-all checklist. Each step must be justified by the specific properties of the data and the model being used. The steps that were clearly necessary and correct — removing the 10 invalid Sex codes, fixing the 172 hidden-zero Cholesterol values, and correcting the 13 negative Oldpeak values — were genuinely important for data integrity. They make sure the model is trained on honest measurements. But applying IQR capping and SMOTE without thinking about whether they are appropriate for this specific situation ended up hurting performance rather than helping it.

5.2 Comparing the Three Models

After tuning, all three models performed within a narrow range of each other. The SVM achieved the highest test F1 (0.854), the Random Forest had the highest ROC-AUC (0.919) and the best cross-validation F1 (0.869), and the MLP finished last on all metrics (F1: 0.834, AUC: 0.897).

The differences between SVM and Random Forest are smaller than the natural variability you would expect from a 184-sample test set — if you repeated the experiment with a different random split, the ranking might flip. Both models are genuinely strong choices for this problem. The MLP's lower performance is more consistent and understandable: with only 734 training samples, the MLP does not have enough data to fully take advantage of its ability to learn complex non-linear patterns. Simpler models like RF and SVM tend to outperform neural networks on small tabular datasets like this one.

Tuning improved the MLP and barely changed anything else. The Random Forest actually scored slightly lower after tuning (F1: 0.844 vs 0.860 default), which is within test-set noise but confirms the default settings were already well-suited. The SVM's grid search selected exactly the same parameters as the defaults ($C=1$, RBF, $\gamma=\text{scale}$), so its test scores are identical before and after tuning. The MLP was the real beneficiary — its F1 improved modestly (+0.006) and its AUC jumped from 0.863 to 0.897 (+0.034), because the tuner correctly identified that a single hidden layer of 64 neurons generalises better than the default two-layer (64+32) architecture on this small dataset.

5.3 Practical Recommendation

If I had to deploy one model for clinical heart disease screening based on this 12-feature representation, I would recommend the tuned SVM with RBF kernel, $C=1$, and $\gamma=\text{scale}$. Here is why:

Best F1 score (0.854): It had the highest balanced measure of precision and recall on the held-out test data.

Near-best ROC-AUC (0.918): It ranks patients almost as well as the Random Forest (0.919), meaning it is very good at prioritizing high-risk patients.

Fast predictions (0.003 seconds): Once trained, the SVM makes predictions almost instantly, which is important in a clinical setting where results should be available quickly.

Compact and reliable: The SVM model is a single mathematical function (once the support vectors are found), making it straightforward to understand and deploy.

That said, the Random Forest is an excellent runner-up and might actually be preferred in a real clinical context because it can produce a feature importance ranking. This ranking helps doctors understand which measurements matter most for a particular patient's prediction, which is crucial for trust and explainability in medical applications.

The MLP, while competitive, is not the best choice for this problem. Its advantage — learning complex deep patterns — is only realized with much larger datasets. With 918 rows, we do not have enough data to train a meaningful neural network that clearly outperforms simpler methods.

6. Conclusion

In this assignment, we built a complete machine learning pipeline for predicting heart disease, starting from raw data exploration and ending with a comparison of three tuned classification models. The work produced several important findings that go beyond just getting good accuracy numbers.

The most important finding from Part 1 is that good preprocessing requires careful thinking, not just following a template. We correctly identified and fixed three types of data quality problems that were specific to this dataset: invalid Sex codes, impossible zero values in Cholesterol and RestingBP, and non-physical negative Oldpeak values. These corrections were essential for making the data honest and reliable. However, we also applied two standard preprocessing steps — IQR outlier capping and SMOTE — that ended up being harmful rather than helpful for the Random Forest model. The IQR capping removed clinically extreme values that were actually strong disease indicators, and SMOTE added unnecessary artificial samples to a dataset that was only mildly imbalanced. This taught us that every preprocessing decision should be motivated by the specific characteristics of the data and the model, not applied automatically.

From Part 2, we learned that hyper-parameter tuning matters most for MLP on this dataset — it produced the largest improvement in AUC (+0.034). For SVM, the grid search simply confirmed the default settings were already optimal, producing no change in test scores. For Random Forest, tuning produced a marginally lower test F1, though the difference is within noise; the cross-validation F1 of 0.869 confirms it is still the strongest model by that measure.

Based on the combination of predictive performance, speed, and computational cost, we recommend the tuned SVM as the best model for this task. The Random Forest is the best choice when model interpretability is needed, such as when doctors want to understand which features contributed most to a specific prediction.

All of the experiments in this assignment can be reproduced exactly by running the notebook `Assignment1_Heart_Disease.ipynb` with the same `Heart_DataSet.csv` file. The random seed is set to 42 throughout, and the train/test split ratio is 80/20 with stratification.

References

- [1] Janosi, A., Steinbrunn, W., Pfisterer, M., & Detrano, R. (1989). Heart Disease [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C52P4X>
- [2] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [3] Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
- [4] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
- [5] Cortes, C., & Vapnik, V. (1995). Support-Vector Networks. *Machine Learning*, 20(3), 273–297.